Recursion

COMP 182 Lecture 04

Recursive methods in Java

Tracing calls, proving termination, and comparing costs

A directory contains smaller directories

To find a word in every file, apply the same search to each subdirectory.

Entry	Work to perform
notes.txt	Search this file
lectures/	Search every entry inside this directory
projects/	Search every entry inside this directory
An empty directory	Return immediately

The task stays the same while the directory being searched becomes smaller.

The structure of a recursive algorithm

A method solves a smaller instance of its own problem.

Part	Question it answers
Input contract	Which inputs are valid?
Base case	When can the method finish without another call?
Recursive step	Which smaller instance should it solve?
Result or action	How does this call use the smaller result?

A correct recursive call must move toward a reachable base case.

Delegating a smaller countdown

Each person prints one number and asks someone else to continue.



Repeated task: print n. Smaller problem: count down from n – 1.

Java: Countdown.java

The countdown reaches its base case

The person handling 1 finishes without passing the task to another person.



At n = 1, print 1 and return without another recursive call.

Java: Countdown.java

Countdown calls and returns

For countdown(3), each call handles one number and delegates the rest.

Call	Action	Next step
countdown(3)	Print 3	Call countdown(2)
countdown(2)	Print 2	Call countdown(1)
countdown(1)	Print 1	Return without another call

Return order: countdown(1), countdown(2), countdown(3)

Each call keeps its place until the smaller call finishes.

Java: Countdown.java

Why recursion terminates

The base case and progress rule must work together for every valid input.

Example	Quantity that gets smaller	Stopping condition
Countdown	Positive value n	n = 1
Binary search	Search interval length	Interval is empty
Hanoi	Number of disks	n = 0
Euclid’s GCD	Second nonnegative argument	Second argument is 0

A base case does not help if the recursive step never reaches it.

Java: Countdown.java

Countdown in Java

Input: n ≥ 1. The method prints n down through 1.

```
static void countdown(int n) {
    if (n < 1) throw new IllegalArgumentException();
    if (n == 1) {
        System.out.println(1);
        return;
    }
    System.out.println(n);
    countdown(n - 1);
}
```

Time O(n). Maximum call-stack space O(n).

Java: Countdown.java

Factorial returns a value

0! = 1. For n > 0, n! = n × (n − 1)!

```
static long factorial(int n) {
    if (n < 0 || n > 20)
        throw new IllegalArgumentException();
    if (n == 0) return 1;
    return n * factorial(n - 1);
}
```

factorial(4) = 4 × 3 × 2 × 1 × 1 = 24

Time O(n). Stack O(n). The pending multiplication happens on return.

Java: Factorial.java

Writing a string backward

Print the last remaining character, then handle the shorter prefix.

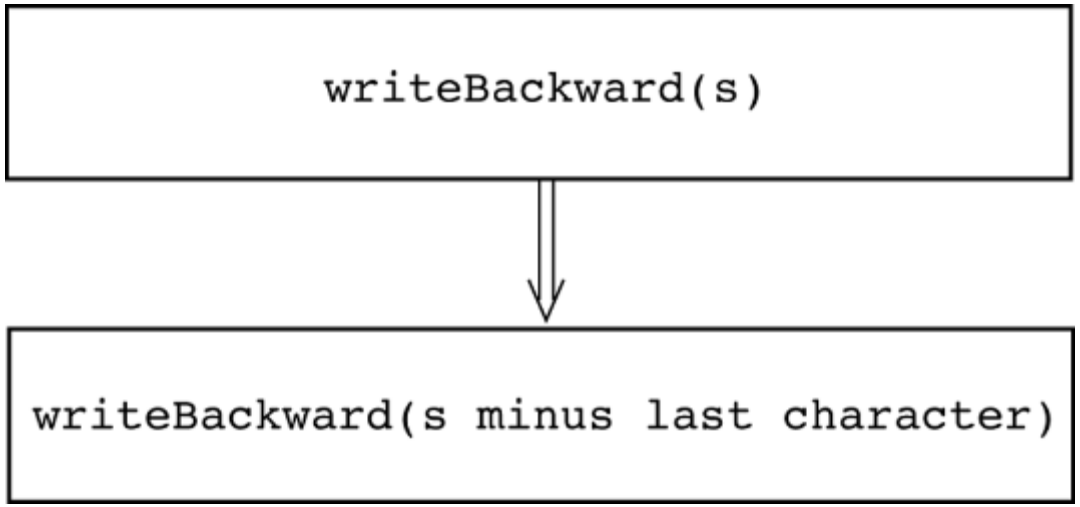
Call index i	Printed character	Remaining prefix
3	E	COD
2	D	CO
1	O	C
0	C	Empty
−1	Nothing	Base case

writeBackward("CODE", 3) prints EDOC.

Java: ReversePrint.java

A smaller prefix without copying substrings

The index marks the last character still to be printed.




```
static void writeBackward(String s, int i) {
    if (i < 0) return;
    System.out.print(s.charAt(i));
    writeBackward(s, i - 1);
}
```

Call: writeBackward(s, s.length() - 1);

For n characters: O(n) time and O(n) call-stack space.

Java: ReversePrint.java

Fibonacci numbers

Each value after the first two is the sum of its two predecessors.

F(0) = 0 F(1) = 1

$F(n) = F(n - 1) + F(n - 2)$, for $n \geq 2$

n	0	1	2	3	4	5	6	7	8	9	10	11
F(n)	0	1	1	2	3	5	8	13	21	34	55	89

Two base cases are necessary: F(0) and F(1).

Java: Fibonacci.java

Naive Fibonacci repeats subproblems

The method is short, but the same smaller values are computed many times.

```
static long fib(int n) {
    if (n < 0 || n > 92)
        throw new IllegalArgumentException();
    if (n < 2) return n;
    return fib(n - 1) + fib(n - 2);
}
```

fib(5) computes fib(3) twice.

Memoization stores earlier results.

Memoized or iterative time: O(n).

Use n ≤ 10 for tracing. long stores values through F(92).

Naive time $O(2^n)$, an upper bound. Maximum stack depth O(n).

Java: Fibonacci.java

A palindrome compares matching ends

Assume the input already has the case and spacing you want to compare.

```
static boolean isPalindrome(String s,
                            int left, int right) {
    if (left >= right) return true;
    return s.charAt(left) == s.charAt(right)
        && isPalindrome(s, left + 1, right - 1);
}
```

"madam" gives true. "robot" gives false.

Each call removes two characters from consideration. Worst case O(n).

Java: Palindrome.java

Recursive binary search

Sorted array: [3, 8, 12, 17, 24, 31, 40]. Target: 24.

```
static int binarySearch(int[] a, int target,
                        int low, int high) {
    if (low > high) return -1;
    int mid = low + (high - low) / 2;
    if (a[mid] == target) return mid;
    if (target < a[mid])
        return binarySearch(a, target, low, mid - 1);
    return binarySearch(a, target, mid + 1, high);
}
```

low	high	mid	a[mid]
0	6	3	17
4	6	5	31
4	4	4	24

Target 24: found at index 4

Call: binarySearch(a, 24, 0, a.length - 1);

Worst-case time O(log n). Recursive stack O(log n).

Java: RecursiveBinarySearch.java

The Tower of Hanoi

Move n disks from peg A to peg B, using peg C as spare storage.

All disks start on A, with the smallest disk on top.

Move only one disk at a time.

Only the top disk on a peg may move.

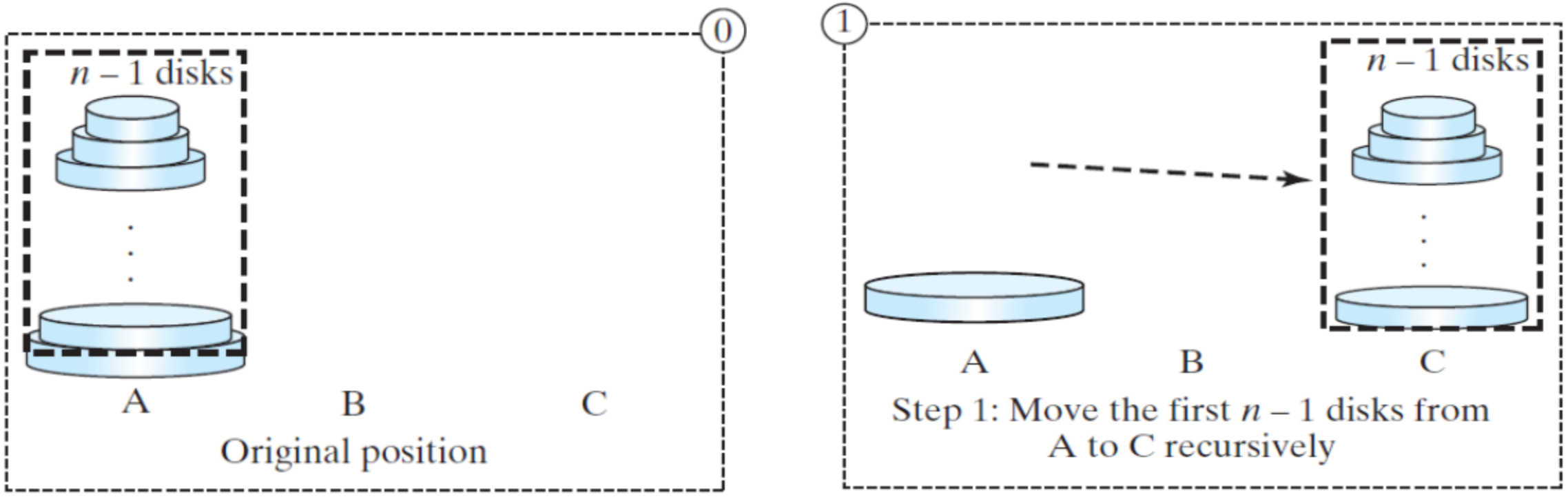
A larger disk may never sit on a smaller disk.

Question: how many moves are necessary for three disks?

Java: TowerOfHanoi.java

Hanoi: clear the largest disk

Move the n − 1 smaller disks from A to C, using B as spare storage.

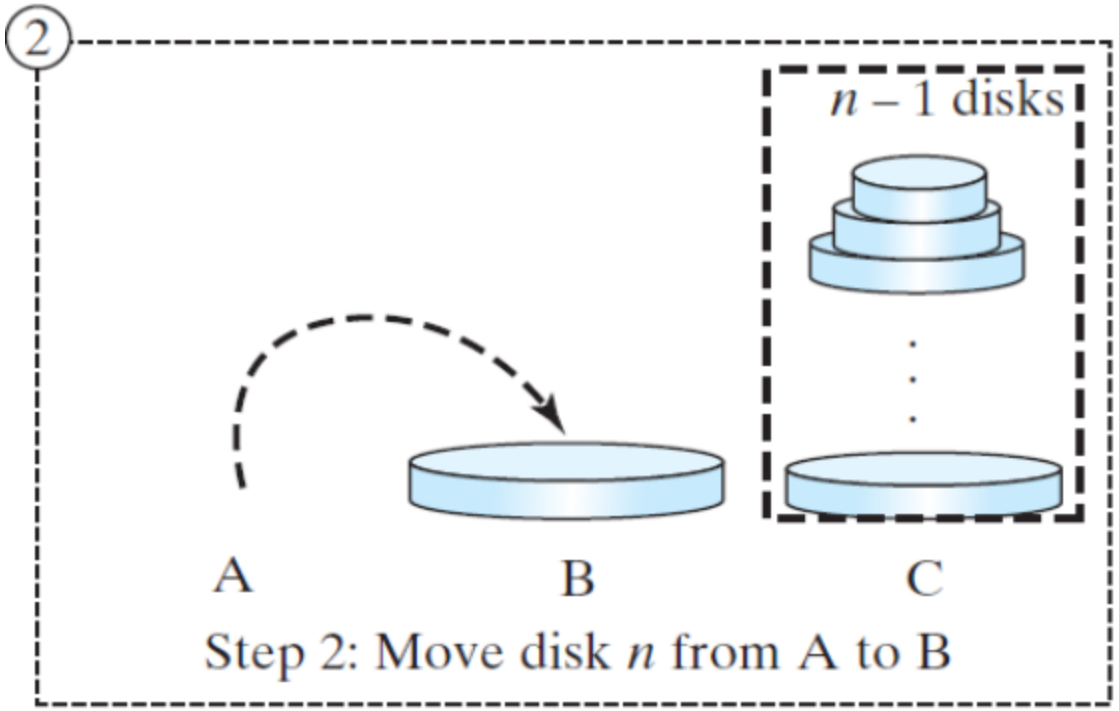


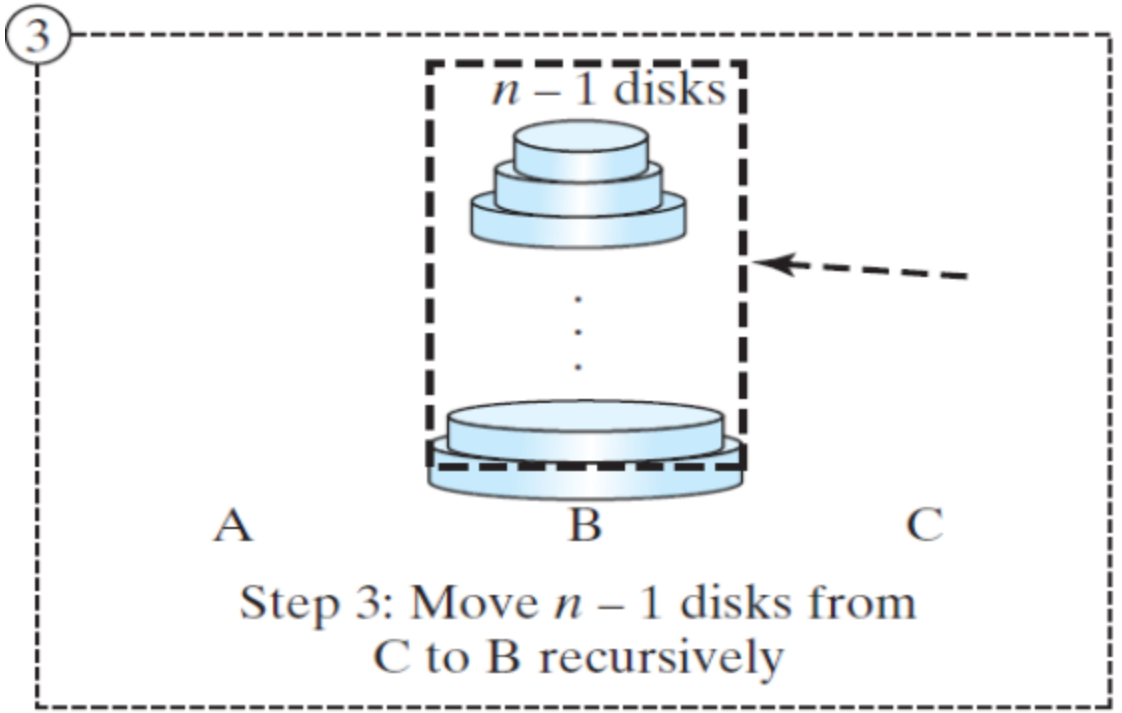
The largest disk stays on A until every smaller disk has moved away.

Java: TowerOfHanoi.java

Hanoi: move disk n and finish

Move the largest disk to B, then transfer the n − 1 smaller disks from C to B.





Two recursive transfers surround one direct move of the largest disk.

Java: TowerOfHanoi.java

Hanoi in Java

Call hanoi(3, 'A', 'B', 'C'). The n = 0 case makes no moves.

```
static void hanoi(int n, char from,
                  char to, char spare) {
    if (n < 0) throw new IllegalArgumentException();
    if (n == 0) return;
    hanoi(n - 1, from, spare, to);
    System.out.println(from + " to " + to);
    hanoi(n - 1, spare, to, from);
}
```

Moves: $M(n) = 2M(n - 1) + 1 = 2^n - 1$

Time $O(2^n)$. Maximum call-stack space $O(n)$.

Java: TowerOfHanoi.java

Greatest common divisor

Euclid's algorithm can be written with a loop or with recursion.

Inputs	GCD
2 and 3	1
2 and 10	2
25 and 35	5
27 and 18	9
0 and 18	18

Brute-force candidate scan: O(min(a, b)) for positive inputs.

Euclid uses gcd(a, b) = gcd(b, a mod b), while b > 0.

Java: GcdIterative.java

Java: GcdRecursive.java

Euclid with a loop

Check b before computing a remainder, so zero inputs are safe.

```
static long gcdIter(int m, int n) {
    long a = Math.abs((long) m);
    long b = Math.abs((long) n);
    while (b != 0) {
        long remainder = a % b;
        a = b;
        b = remainder;
    }
    return a;
}
```

Positive inputs: O(log min(a, b)) time. Auxiliary space O(1).

Java: GcdIterative.java

Euclid with recursion

The second argument shrinks to zero. The first argument is then the GCD.

```
static long gcd(int m, int n) {
    return gcdPositive(Math.abs((long) m),
                       Math.abs((long) n));
}

static long gcdPositive(long a, long b) {
    if (b == 0) return a;
    return gcdPositive(b, a % b);
}
```

a	b	a mod b
27	18	9
18	9	0
9	0	Stop

gcd(27, 18) returns 9

Positive inputs: O(log min(a, b)) time and recursive stack space.

Java: GcdRecursive.java

When recursion matches the problem

Recursion is useful when the problem already has a recursive structure.

Problem	Smaller problem	Work in this call
Directory search	Each subdirectory	Process files and subdirectories
Factorial	Factorial of n − 1	Multiply the returned result by n
Binary search	One half of the interval	Compare and select that half
Hanoi	Transfer n − 1 disks	Move the largest disk between calls

Choose recursion when its structure makes the algorithm easier to reason about.

Time and call-stack costs

Worst-case bounds for the examples in this lecture.

Algorithm	Time	Call-stack space
Countdown / factorial	O(n)	O(n)
String reversal / palindrome	O(n)	O(n)
Binary search	O(log n)	O(log n)
Naive Fibonacci	$O(2^n)$, upper bound	O(n)
Hanoi	$O(2^n)$	O(n)
Recursive Euclid (a, b > 0)	O(log min(a, b))	O(log min(a, b))

Deep recursion can exhaust the stack. Repeated subproblems can waste time.

A checklist for correct recursion

Use the input contract, stopping rule, and progress measure to check a solution.

State the valid inputs and handle boundary cases.

Make the base case explicit and reachable.

Show why every recursive call makes progress.

Use the recursive result and account for stack depth.

Quick checks: factorial(0), an empty binary search, Hanoi with 3 disks, gcd(27, 18)

Trace one small input through both the calls and the returns.